EMBEDDED SYSTEMS & ROBOTICS ENGINEERING PORTFOLIO

Practical Engineering Portfolio

Design, Firmware Implementation, Electrical Schematics, and Safety Testing for Autonomous Robotics & IoT Systems

AUTHOR PROFILE
Student Maker & Aspiring Embedded Systems Engineer

CORE SPECIALIZATIONS
Robotics, Arduino C/C++, ESP8266 IoT, Circuit Prototyping

FEATURED PROJECTS
1. Obstacle-Avoiding Robot Car
2. IoT Plant-Watering System

DOCUMENT SCOPE
Code, Wiring, Diagnostics, Reports & Learning Journey

1. Personal Engineering Journey: Student Inventor to Future Engineer

1.1 Introduction & The Beginning

My journey into technology, electronics, and robotics began in Class 9. Rather than remaining confined to textbook equations and theoretical concepts, I felt an intense drive to build real physical machines. At that time, I did not possess advanced technical knowledge—I was actively learning the fundamentals of computer programming, basic circuit laws, and mechanical linkages from scratch.

Working with accessible, affordable components like the Arduino (ATmega328P) and standard hobby gearboxes, I began experimenting with what was available to me. I quickly discovered that the physical world is far more unforgiving than software simulation: motors draw unpredictable surge currents, mechanical balance shifts dynamically under acceleration, and sensors encounter real-world acoustic and electrical noise.

1.2 Financial & Learning Realities as an Advantage

As a student with limited financial resources, purchasing high-end computing platforms, custom laser-cut chassis, or industrial PLC modules was out of reach. When a component failed or an IC burned out, replacement was not trivial.

Rather than seeing this as a handicap, I treated limited resources as an engineering discipline. It forced me to understand the root causes of every failure:

- I learned why inductive flyback diodes are mandatory when switching DC motors and relays.
- I learned why common-ground reference lines must tie logic boards to motor drivers.
- I learned how to write defensive firmware with watchdogs and hardware timeouts rather than relying on perfect operating conditions.

1.3 Core Technical Interests & Long-Term Roadmap

My technical focus centers on the intersection of physical hardware and low-level software:

- **Embedded Systems:** Bare-metal C/C++, timer interrupts, PWM generation, and register-level peripheral control.
- **IoT & Distributed Automation:** ESP8266/ESP32 network stacks, cloud telemetry, and fail-safe remote actuation.
- **Custom PCB Design:** Transitioning from breadboard jumper wires to production-grade 2-layer and 4-layer PCBs using KiCad.
- **3D Prototyping:** Parametric CAD modeling (OpenSCAD/FreeCAD) for custom motor brackets and robotic chassis.
- **Humanoid Robotics:** Long-term aspiration to develop multi-degree-of-freedom articulated robotic systems, starting from servo kinematics and progressing toward dynamic balance control.

---

2. Project 1: Arduino Obstacle-Avoiding Robot Car

2.1 Executive Summary & Working Principle

The Obstacle-Avoiding Robot Car is a two-wheel-drive (2WD) autonomous mobile platform developed to navigate unknown indoor environments without human intervention. The system utilizes an Arduino Uno/Nano microcontroller, an HC-SR04 ultrasonic distance sensor mounted on an SG90 micro-servo, and an L298N dual H-bridge motor driver.

Autonomous Navigation State Machine

1. **Forward Navigation:** Ultrasonic sensor aimed at 90° center. If forward clearance exceeds 20 cm, both DC gear motors drive forward at PWM speed 180.
2. **Obstacle Trigger:** When an obstacle enters the 20 cm threshold, forward motors immediately stop to halt kinetic momentum.
3. **Clearance Back-off:** The vehicle reverses for 300 ms to create turning room, followed by a 150 ms settle delay.
4. **Environmental Scan:** The servo rotates to 150° (Left) to measure clearance, then sweeps to 30° (Right) to measure clearance, before returning to 90° center.
5. **Decision & Turn:** Clearance distances are compared. The vehicle executes an open-loop pivot turn toward the side with greater clear space for 550 ms, then resumes forward motion.

2.2 Hardware Interconnect & Pin Mapping

Subsystem	Module Pin	Arduino Pin	Electrical Characteristics / Logic Role
Ultrasonic Sensor (HC-SR04)	TRIG	Pin 11	Digital Output: 10 µs trigger acoustic burst
	ECHO	Pin 12	Digital Input: Pulse width reflects acoustic echo return
Pan Servo (SG90)	Signal	Pin 3	PWM Output: 50 Hz pulse-width position control (30° to 150°)
Right Motor (L298N Channel A)	ENA	Pin 5	Timer PWM Output: Speed control (0–255, set to 180)
	IN1	Pin 7	Digital Output: Direction polarity A
	IN2	Pin 8	Digital Output: Direction polarity B
Left Motor (L298N Channel B)	ENB	Pin 6	Timer PWM Output: Speed control (0–255, set to 180)
	IN3	Pin 9	Digital Output: Direction polarity A
	IN4	Pin 10	Digital Output: Direction polarity B
Power Architecture	12V & GND	External / GND	7.4V (2S Li-ion) to L298N. **Common Ground to Arduino GND is mandatory.**

2.3 Direction Control Truth Table

Command	ENA (D5)	IN1 (D7)	IN2 (D8)	ENB (D6)	IN3 (D9)	IN4 (D10)	Right Wheel	Left Wheel	Chassis Movement
Stop	0	LOW	LOW	0	LOW	LOW	Brake / Off	Brake / Off	Stationary
Forward	180	HIGH	LOW	180	HIGH	LOW	Forward	Forward	Straight Forward
Reverse	180	LOW	HIGH	180	LOW	HIGH	Reverse	Reverse	Straight Backward
Pivot Left	180	HIGH	LOW	180	LOW	HIGH	Forward	Reverse	Counter-Clockwise Spin
Pivot Right	180	LOW	HIGH	180	HIGH	LOW	Reverse	Forward	Clockwise Spin

2.4 Cleaned & Security-Hardened Source Code (`robot.ino`)

```
/*
  Project: Arduino Obstacle-Avoiding Robot Car
  Platform: Arduino Uno / Nano (ATmega328P) | License: MIT
*/
#include <Servo.h>
#include <NewPing.h>

const uint8_t PIN_TRIG = 11;
const uint8_t PIN_ECHO = 12;
const uint8_t PIN_SERVO = 3;

const uint8_t PIN_ENA  = 5;   // Right Motor Speed (PWM)
const uint8_t PIN_IN1  = 7;   // Right Motor Direction 1
const uint8_t PIN_IN2  = 8;   // Right Motor Direction 2
const uint8_t PIN_ENB  = 6;   // Left Motor Speed (PWM)
const uint8_t PIN_IN3  = 9;   // Left Motor Direction 1
const uint8_t PIN_IN4  = 10;  // Left Motor Direction 2

const unsigned int MAX_DISTANCE_CM       = 200;
const unsigned int OBSTACLE_THRESHOLD_CM = 20;
const uint8_t MOTOR_SPEED                = 180;

const unsigned long DURATION_REVERSE_MS  = 300;
const unsigned long DURATION_TURN_MS     = 550;
const unsigned long DURATION_SETTLE_MS   = 150;

const uint8_t SERVO_CENTER_DEG = 90;
const uint8_t SERVO_LEFT_DEG   = 150;
const uint8_t SERVO_RIGHT_DEG  = 30;
const unsigned long SERVO_DELAY_MS = 400;

NewPing sonar(PIN_TRIG, PIN_ECHO, MAX_DISTANCE_CM);
Servo scanServo;

unsigned int getDistanceCm();
unsigned int scanDirection(uint8_t angle);
void moveForward(uint8_t speed);
void moveBackward(uint8_t speed);
void turnLeft(uint8_t speed);
void turnRight(uint8_t speed);
```

```
void stopMotors();

void setup() {
  pinMode(PIN_ENA, OUTPUT);
  pinMode(PIN_IN1, OUTPUT);
  pinMode(PIN_IN2, OUTPUT);
  pinMode(PIN_ENB, OUTPUT);
  pinMode(PIN_IN3, OUTPUT);
  pinMode(PIN_IN4, OUTPUT);

  stopMotors();

  scanServo.attach(PIN_SERVO);
  scanServo.write(SERVO_CENTER_DEG);
  delay(1000);
}

void loop() {
  unsigned int frontDistance = getDistanceCm();

  if (frontDistance > 0 && frontDistance <= OBSTACLE_THRESHOLD_CM) {
    stopMotors();
    delay(DURATION_SETTLE_MS);

    moveBackward(MOTOR_SPEED);
    delay(DURATION_REVERSE_MS);
    stopMotors();
    delay(DURATION_SETTLE_MS);

    unsigned int leftDistance  = scanDirection(SERVO_LEFT_DEG);
    unsigned int rightDistance = scanDirection(SERVO_RIGHT_DEG);

    scanServo.write(SERVO_CENTER_DEG);
    delay(200);

    if (leftDistance >= rightDistance) {
      turnLeft(MOTOR_SPEED);
      delay(DURATION_TURN_MS);
    } else {
      turnRight(MOTOR_SPEED);
      delay(DURATION_TURN_MS);
    }

    stopMotors();
    delay(DURATION_SETTLE_MS);
  } else {
    moveForward(MOTOR_SPEED);
  }
  delay(40);
}

unsigned int getDistanceCm() {
  delay(30);
  return sonar.ping_cm();
}

unsigned int scanDirection(uint8_t angle) {
  scanServo.write(angle);
  delay(SERVO_DELAY_MS);
  unsigned int dist = getDistanceCm();
  if (dist == 0) {
    dist = MAX_DISTANCE_CM; // Treat out-of-range echo as clear corridor
  }
  return dist;
}

void moveForward(uint8_t speed) {
```

```
  digitalWrite(PIN_IN1, HIGH); digitalWrite(PIN_IN2, LOW); analogWrite(PIN_ENA, speed);
  digitalWrite(PIN_IN3, HIGH); digitalWrite(PIN_IN4, LOW); analogWrite(PIN_ENB, speed);
}

void moveBackward(uint8_t speed) {
  digitalWrite(PIN_IN1, LOW); digitalWrite(PIN_IN2, HIGH); analogWrite(PIN_ENA, speed);
  digitalWrite(PIN_IN3, LOW); digitalWrite(PIN_IN4, HIGH); analogWrite(PIN_ENB, speed);
}

void turnLeft(uint8_t speed) {
  digitalWrite(PIN_IN1, HIGH); digitalWrite(PIN_IN2, LOW); analogWrite(PIN_ENA, speed);
  digitalWrite(PIN_IN3, LOW); digitalWrite(PIN_IN4, HIGH); analogWrite(PIN_ENB, speed);
}

void turnRight(uint8_t speed) {
  digitalWrite(PIN_IN1, LOW); digitalWrite(PIN_IN2, HIGH); analogWrite(PIN_ENA, speed);
  digitalWrite(PIN_IN3, HIGH); digitalWrite(PIN_IN4, LOW); analogWrite(PIN_ENB, speed);
}

void stopMotors() {
  digitalWrite(PIN_IN1, LOW); digitalWrite(PIN_IN2, LOW);
  digitalWrite(PIN_IN3, LOW); digitalWrite(PIN_IN4, LOW);
  analogWrite(PIN_ENA, 0); analogWrite(PIN_ENB, 0);
}
```

2.5 Troubleshooting & Failure Diagnostics

Observed Symptom	Root Cause	Bench Diagnostic Step	Resolution
Arduino resets when motors start	Voltage dip on 5V logic rail due to motor inrush current	Measure 5V rail with multimeter while motor engages	Separate motor battery supply from Arduino 5V; add 470 µF decoupling capacitor
Servo twitches or stalls	Horn binds against chassis at 0° or 180° limits	Inspect physical clearance at extreme angles	Constrain software angles to 30° and 150°
Car turns continuously	Echo wire disconnected or sensor tilted downward toward floor	Check Serial Monitor print of distance values	Reseat Pin 12 jumper wire; tilt sensor upward ~3°
Turn angle < 90°	Battery voltage drop or high carpet friction	Compare turn on tile vs carpet; test battery under load	Increase turn duration or recharge 2S Li-ion battery pack

3. Project 2: ESP8266 Automatic Plant-Watering System with Blynk IoT

3.1 Executive Summary & Architectural Scope

Developed during Class 10, the Automatic Plant-Watering System expands embedded programming into IoT cloud automation. Utilizing a NodeMCU ESP8266 development board, a soil moisture sensor, a 5V optocoupled relay, and

a mini 5V DC submersible pump, the system monitors potting soil hydration in real time and automatically runs irrigation cycles.

> **Hardware Integrity & Verification Notice**
>
> This project documentation reflects strictly verified physical components: NodeMCU ESP8266, soil moisture sensor, 5V relay module, and DC pump. **No ambient temperature or humidity sensors (DHT11/DHT22) are installed or claimed.** Furthermore, while original project logs were reviewed, the user's initial draft sketch was missing from provided files. Below is the production-grade, fail-safe architecture designed for their physical hardware.

3.2 Critical Safety & Hardware Engineering Solutions

- **Active-LOW Boot Protection:** Budget 5V relay modules energize when the control pin is driven LOW. During microcontroller boot, floating GPIOs can trigger momentary relay clicks. The firmware forces `digitalWrite(PIN_RELAY, HIGH)` *before* calling `pinMode(PIN_RELAY, OUTPUT)` to guarantee a silent, safe boot.
- **Galvanic Probe Corrosion Mitigation:** Continual DC current through moist soil causes rapid electrolytic erosion of copper sensor forks. The firmware powers the sensor via a digital GPIO pin (D5) only for 20 ms during measurement intervals every 3 seconds, cutting electrolysis by >99%.
- **Inductive Flyback Snubber:** A reverse-biased 1N4007 diode across pump terminals suppresses inductive voltage spikes, preventing ESP8266 Wi-Fi disconnects.
- **Flood-Prevention Watchdog:** If the sensor becomes dislodged from the soil, automatic irrigation automatically aborts after 10 seconds of continuous pumping, followed by a mandatory 60-second soak cooldown.

3.3 Cleaned Source Code (`plant_watering.ino`)

```
/*
  Project: ESP8266 Automatic Plant-Watering System with Blynk IoT
  Target: NodeMCU ESP8266 | License: MIT
*/
#if __has_include("config.h")
  #include "config.h"
#else
  #define BLYNK_TEMPLATE_ID   "YOUR_BLYNK_TEMPLATE_ID"
  #define BLYNK_TEMPLATE_NAME "YOUR_BLYNK_TEMPLATE_NAME"
  #define BLYNK_AUTH_TOKEN    "YOUR_BLYNK_AUTH_TOKEN"
  #define WIFI_SSID           "YOUR_WIFI_SSID"
  #define WIFI_PASSWORD       "YOUR_WIFI_PASSWORD"
#endif

#define BLYNK_PRINT Serial
#include <ESP8266WiFi.h>
#include <BlynkSimpleEsp8266.h>

const uint8_t PIN_SOIL_ANALOG = A0;
const uint8_t PIN_RELAY       = D1;  // GPIO5 (Safe boot pin)
const uint8_t PIN_SENSOR_VCC  = D5;  // GPIO14 (Duty-cycled probe power)

const uint8_t RELAY_ON  = LOW;   // Active-LOW
const uint8_t RELAY_OFF = HIGH;

const int CALIBRATION_AIR_DRY   = 900;
const int CALIBRATION_WATER_WET = 350;

int autoMoistureThreshold = 35; // Default watering threshold: 35%

const unsigned long MAX_PUMP_RUN_TIME_MS = 10000; // 10s safety limit
```

```
const unsigned long PUMP_COOLDOWN_MS     = 60000; // 1 min cooldown
const unsigned long SENSOR_READ_INTERVAL = 3000;

bool isManualMode = false;
bool manualPumpCommand = false;
bool isPumpRunning = false;
unsigned long pumpStartTime = 0;
unsigned long pumpStopTime = 0;
int currentMoisturePercent = 0;

BlynkTimer timer;

void readSensorsAndEvaluate();
void setPumpState(bool turnOn, const char* reason);
void updateBlynkTelemetry();

BLYNK_WRITE(V2) {
  manualPumpCommand = (param.asInt() == 1);
  if (isManualMode) {
    setPumpState(manualPumpCommand, manualPumpCommand ? "Manual ON" : "Manual OFF");
  }
}

BLYNK_WRITE(V3) {
  isManualMode = (param.asInt() == 1);
  if (!isManualMode && isPumpRunning) {
    setPumpState(false, "Switched to Auto Mode");
  }
}

BLYNK_WRITE(V4) {
  autoMoistureThreshold = param.asInt();
}

void setup() {
  Serial.begin(115200);

  // Safe boot order: write HIGH before configuring OUTPUT
  digitalWrite(PIN_RELAY, RELAY_OFF);
  pinMode(PIN_RELAY, OUTPUT);
  digitalWrite(PIN_RELAY, RELAY_OFF);

  pinMode(PIN_SENSOR_VCC, OUTPUT);
  digitalWrite(PIN_SENSOR_VCC, LOW);

  Blynk.begin(BLYNK_AUTH_TOKEN, WIFI_SSID, WIFI_PASSWORD);

  timer.setInterval(SENSOR_READ_INTERVAL, readSensorsAndEvaluate);
  timer.setInterval(2000L, updateBlynkTelemetry);
}

void loop() {
  Blynk.run();
  timer.run();

  if (isPumpRunning && (millis() - pumpStartTime >= MAX_PUMP_RUN_TIME_MS)) {
    setPumpState(false, "Safety Timeout Exceeded");
  }

  if (WiFi.status() != WL_CONNECTED && isPumpRunning) {
    setPumpState(false, "Wi-Fi Lost Safety Shutoff");
  }
}

void readSensorsAndEvaluate() {
  digitalWrite(PIN_SENSOR_VCC, HIGH);
  delay(20);
```

```
  int rawValue = analogRead(PIN_SOIL_ANALOG);
  digitalWrite(PIN_SENSOR_VCC, LOW);

  int mapped = map(rawValue, CALIBRATION_AIR_DRY, CALIBRATION_WATER_WET, 0, 100);
  currentMoisturePercent = constrain(mapped, 0, 100);

  if (!isManualMode) {
    bool cooledDown = (millis() - pumpStopTime >= PUMP_COOLDOWN_MS);
    if (currentMoisturePercent < autoMoistureThreshold) {
      if (!isPumpRunning && cooledDown) {
        setPumpState(true, "Auto: Low moisture");
      }
    } else {
      if (isPumpRunning) {
        setPumpState(false, "Auto: Moisture satisfied");
      }
    }
  }
}

void setPumpState(bool turnOn, const char* reason) {
  if (turnOn) {
    digitalWrite(PIN_RELAY, RELAY_ON);
    isPumpRunning = true;
    pumpStartTime = millis();
  } else {
    digitalWrite(PIN_RELAY, RELAY_OFF);
    isPumpRunning = false;
    pumpStopTime = millis();
  }
  Blynk.virtualWrite(V1, isPumpRunning ? 1 : 0);
}

void updateBlynkTelemetry() {
  if (Blynk.connected()) {
    Blynk.virtualWrite(V0, currentMoisturePercent);
    Blynk.virtualWrite(V1, isPumpRunning ? 1 : 0);
  }
}
```

3.4 Blynk IoT 2.0 Datastream Mapping

Virtual Pin	Datastream Name	Type	Range	Associated UI Widget
V0	Soil Moisture	Integer	0 to 100 %	Radial Gauge / Numerical Display
V1	Pump State	Integer	0 (OFF) / 1 (ON)	Green Status LED Widget
V2	Manual Pump Switch	Integer	0 (OFF) / 1 (ON)	Push Button / Toggle Switch
V3	Control Mode	Integer	0 (Auto) / 1 (Manual)	Segmented Selector Switch
V4	Auto Threshold	Integer	10 to 70 %	Horizontal Slider Widget

4. Security, Secrets Hygiene & Privacy Audit

4.1 Credentials Isolation Architecture

In embedded and IoT open-source repositories, committing hardcoded Wi-Fi passwords and cloud authentication tokens represents a critical vulnerability. This project implements a strict separation of configuration and code:

Credential / Data Type	Git Repository Handling	Local Device Handling
Blynk Auth Token	Excluded. Protected by `.gitignore` rule matching `config.h`.	User populates private `src/config.h` on local workstation.
Wi-Fi SSID & Password	Excluded. Template committed with placeholders in `config.example.h`.	Never committed to any Git branch.
Personal Identifiable Info (PII)	No personal phone numbers, physical addresses, or private emails published.	Author identified purely through professional portfolio branding.
Banking / Financial Data	Zero banking info, IFSC codes, or payment references included.	Strictly separated from technical engineering files.

> **Remediation for Previously Committed Secrets**
>
> If a real Blynk token or Wi-Fi password was previously committed to a Git repository, simply deleting the line and making a new commit **does not remove the secret from Git history**. Anyone can inspect historical commits. In such cases, the token must be immediately invalidated and regenerated via the Blynk.Console Web UI.

5. GitHub Portfolio Structure & Deployment Guide

5.1 Recommended Repository Structure

```
├── README.md                                # Master GitHub Profile README
├── portfolio/
│   └── README.md                            # Personal Engineering Story & Journey
├── robot-obstacle-avoidance/
│   ├── README.md                            # Project overview, pinout, & upload guide
│   ├── LICENSE                              # MIT Open-Source License
│   ├── .gitignore                           # Arduino build & temp file ignores
│   ├── src/
│   │   ├── robot.ino                        # Cleaned & commented source code
│   │   └── original_backup/
│   │       └── robot_original_backup.ino    # Exact original code preserved
│   ├── docs/
│   │   ├── PROJECT_REPORT.md                # Development story, hurdles & lessons
│   │   ├── WIRING.md                        # Schematics, common GND & power rules
│   │   ├── TROUBLESHOOTING.md               # Diagnostic matrix & bench debugging
│   │   └── TESTING.md                       # Pre-power & floor-run checklists
│   └── hardware/
│       ├── components.md                    # Bill of materials & trade-offs
│       └── pin_connections.md               # Direction control truth table & pin map
└── automatic-plant-watering/
```

```
    ├── README.md                          # Project overview, architecture & safety
    ├── LICENSE                            # MIT Open-Source License
    ├── .gitignore                         # Secrets protection (.gitignore config.h)
    ├── src/
    │   ├── plant_watering.ino             # Fail-safe ESP8266 + Blynk sketch
    │   └── config.example.h               # Sanitized credentials template
    ├── docs/
    │   ├── PROJECT_REPORT.md              # Development narrative & iteration history
    │   ├── WIRING.md                      # Inductive isolation & power architecture
    │   ├── BLYNK_SETUP.md                 # Blynk.Console template & datastreams
    │   ├── TROUBLESHOOTING.md             # Calibration & relay diagnostic matrix
    │   └── TESTING.md                     # Dry & wet commissioning checklists
    └── hardware/
        └── components.md                  # Sensor evaluation (resistive vs capacitive)
```

5.2 Step-by-Step Terminal Git Upload Instructions

To publish the repositories to GitHub, execute the following commands in Git Bash or terminal:

```
# 1. Initialize and push the Robot Car repository
cd robot-obstacle-avoidance
git init
git add .
git commit -m "feat: initial commit of verified obstacle-avoiding robot car"
git branch -M main
git remote add origin https://github.com/<YOUR_USERNAME>/robot-obstacle-avoidance.git
git push -u origin main

# 2. Initialize and push the Plant-Watering repository
cd ../automatic-plant-watering
# VERIFY: Run 'git status' to ensure config.h is NOT staged!
git status
git init
git add .
git commit -m "feat: initial commit of ESP8266 IoT plant-watering system with Blynk 2.0"
git branch -M main
git remote add origin https://github.com/<YOUR_USERNAME>/automatic-plant-watering.git
git push -u origin main
```

5.3 Release Tagging & Commit Conventions

Maintaining a professional GitHub presence requires consistent version control practices beyond basic uploads:

- **Conventional Commits:** Structure commit messages with clear semantic prefixes: `feat:` for working capabilities, `fix:` for corrected circuit or software bugs, and `docs:` for wiring and documentation updates.
- **Release Tagging:** Tag milestones with semantic versioning (e.g., `git tag -a v1.0.0 -m "Release v1.0.0: Working prototype"`). This allows employers and mentors to track the exact evolution of hardware iterations.
- **Hardware Binary Assets vs. Source:** Distinguish editable design files (e.g., OpenSCAD `.scad` scripts, KiCad `.kicad_sch` schematics) from compiled output artifacts (`.stl` meshes, Gerber zip packages).

5.4 Photography & Visual Evidence Standards

Employers and reviewers assess hardware competency through real visual proof. In the `images/` directory of each repository, maintain:

- Top-down high-resolution photographs of the physical breadboard / chassis wiring.

- Close-up captures of motor driver jumper configurations and relay modules.
- Screen captures of the Serial Monitor output demonstrating sensor calibration and live telemetry.

---

6. Engineering Skills & Future Learning Roadmap

This portfolio establishes a solid foundation in 8-bit AVR microcontrollers, basic robotics actuation, and cloud IoT telemetry. Below is the structured technical learning roadmap to advance into professional embedded engineering and robotics:

Development Phase	Target Skills & Software	Concrete Milestone Projects
Phase 1: Foundations (Current)	Arduino C/C++, ESP8266, L298N drivers, basic breadboard circuits.	Obstacle-avoiding 2WD robot car; Blynk-connected smart irrigation system.
Phase 2: Custom PCB Design	KiCad 8/9 EDA, IPC-2221A layout rules, trace thermal derating, ground planes.	Design an integrated Arduino shield PCB replacing jumper wires with screw terminals and decoupling caps.
Phase 3: 3D CAD & Prototyping	FreeCAD / OpenSCAD, FDM 3D printing, mechanical fit tolerances.	Design and print parametric sensor brackets, wheel rims, and a balanced unibody robot chassis.
Phase 4: Closed-Loop Control	Quadrature optical encoders, PID motor speed control, MPU-6050 6-axis IMU.	Upgrade robot from open-loop timed turns to encoder-based closed-loop odometry and heading hold.
Phase 5: 32-Bit & RTOS Systems	ESP32 Dual-Core / STM32 Cortex-M, FreeRTOS, hardware timers, DMA.	Multi-tasking sensor hub with asynchronous data logging and low-power sleep modes.
Phase 6: Humanoid Robotics	Multi-DOF servo kinematics, serial bus servos, ROS 2 on Linux, Python.	Build a 6-DOF robotic manipulator arm; progress toward bipedal walking gait mechanics.

> **Summary of Demonstrated Competencies**
>
> • **Embedded C/C++:** Non-blocking state machines, timer PWM control, sensor acquisition.
> • **Hardware Debugging:** Multimeter fault isolation, ground loops, inductive kickback protection, Brownout prevention.
> • **IoT Cloud Integration:** Blynk 2.0 virtual datastreams, Wi-Fi reconnection handling, fail-safe disconnect routines.
> • **Technical Documentation:** Comprehensive wiring guides, diagnostic matrices, and honest development reporting.